scientific reports

Check for updates

OPEN

Enhancing sarcasm detection in sentiment analysis for cyberspace safety using advanced deep learning techniques

Raghu Dhumpati[1], Archana Sasi[2], Shaik Johny Basha[3], Suryanarayana Vadhri[4], P. Saroja[5], Swathi Alluri[6] & Ramesh Vatambeti[7]✉

Social media has become an integral part of daily life, with platforms like Twitter serving as popular outlets for users to share information and express grievances. While social media offers numerous benefits, it can also be misused for cyberbullying, such as insults and harassment. This highlights the importance of detecting and mitigating cyberbullying to ensure a safe online environment. Detecting sarcasm in social media posts is challenging and requires advanced automated systems. To address this, a deep learning-based approach was developed using a Convolutional Neural Network (CNN) for analysis and an Attention Mechanism-based Bidirectional Long Short-Term Memory with Gated Recurrent Unit (AM-BLSTM-GRU) for prediction. Sarcasm detection datasets were gathered from sources like Kaggle and News Headlines. Standard NLP-based auxiliary features were extracted, and an embedded CNN model refined these features into feature vectors. The AM-BLSTM-GRU model then performed sarcasm detection and sentiment classification. The Enhanced Sinogramic Red Deer (ESRD) optimizer was utilized to optimize the classifier parameters effectively. The proposed model outperformed existing deep learning approaches on popular benchmark datasets and evaluation metrics, demonstrating its effectiveness in detecting cyberbullying. This approach was validated using well-known datasets and metrics, confirming its efficacy in identifying and reducing harmful online behaviors.

Keywords Sarcasm detection, Enhanced sinogramic red deer, Twitter data, Sentiment analysis, Auxiliary features

In the age of social media, platforms like Facebook, Instagram, and Twitter have revolutionized the way people communicate, leading to a massive influx of data readily available at our fingertips[1]. One of the most prevalent tasks in data mining from these platforms is sentiment analysis, which aims to determine the polarity of online posts, classifying them as positive, negative, or neutral[2–3]. However, human sentiments are complex, encompassing more nuanced emotions that resist straightforward categorization. Online discourse often features a pseudo-language characterized by casual tone, morphological shortenings, cyberslang, and metaphorical expressions like sarcasm, irony, and humor[4]. The diversity of content is further enriched by cultural variety, country-specific hot topics, hashtags, and the use of native language keyboards[5–6]. Effective sentiment analysis thus requires tackling two distinct natural language processing tasks: real-time emotion recognition and sarcasm detection[7].

Sarcasm, a socially constructed linguistic phenomenon, often conveys deeply ingrained, prejudiced views[8]. It can spread rapidly on social media, where users freely express beliefs, with potential positive or negative effects on others[9]. Despite early studies suggesting that sarcasm enhances creativity and problem-solving, it remains challenging to detect due to its inherent contradictions and the different meanings conveyed by sarcastic

[1]Department of Computer Science and Engineering, Bahrain Polytechnic, Isa Town 3339, Bahrain. [2]Department of CSE, Faculty of Engineering and Technology, Jain University, Kannagapura Rd, Bengaluru 562112, Karnataka, India. [3]Department of CSE, School of Engineering and Sciences, SRM University- AP, Mangalagiri, Andhra Pradesh, India. [4]Department of AI&DS, Lakireddy Bali Reddy College of Engineering, Mylavaram 521230, India. [5]Department of CSE, SRKR Engineering College, Bhimavaram, India. [6]Department of Computer Science and Engineering, Koneru Lakshmaiah Education Foundation, Vaddeswaram, Guntur 522501, India. [7]School of Computer Science and Engineering, VIT-AP University, Vijayawada 522237, India. ✉email: ramesh.v@vitap.ac.in

language compared to the literal text[10– 11]. Many social media users employ sarcasm as a roundabout way to criticize others, complicating the process of accurately understanding and analyzing sentiments[12].

The complexity of sarcasm in online interactions has led researchers to explore various methods for detection[13]. Traditional approaches relied on rule-based and statistical methods, which focused primarily on text significance. However, these methods often fall short of capturing the intricate context and subtlety inherent in sarcastic expressions[14]. Advanced deep learning techniques have emerged as promising solutions, capable of extracting emotions and detecting sarcasm by leveraging both contextual and textual information from platforms like Twitter[15].

Sarcasm detection is essential for combating stress and hostility on social media, and various techniques have been employed to achieve this goal. The lack of spoken tone in social media text makes detecting sarcasm challenging, as vocal intonation is typically a key indicator. Users often use multi-letter words, elongated syllables, and specific terms to convey sarcasm, adding another layer of complexity[16]. However, recent advances in feature extraction methods have significantly improved the performance of machine learning-based sentiment analysis. These methods address the challenges of finding unreliable and ambiguous features by developing hybrid feature extraction models that consider sentence-specific attributes and broader contextual elements[17].

Feature extraction is critical in accurately capturing the sentiment of a post, especially when dealing with implicit features and word relations that convey sentiment, as seen in the example "Although this mobile phone is too heavy, it is a little cheap." Here, implicit features like weight and price and their sentiment-bearing word relations result in an overall neutral mood. To overcome challenges posed by non-standard terminology, an aspect-based approach can group similar features into attributes, facilitating more precise sentiment analysis[18].

The rapid growth of deep learning has revolutionized natural language processing tasks such as text classification and sentiment analysis. Deep learning models mimic neural networks in the human brain, enabling them to learn from long sequences of text and improve performance over time. However, sarcasm detection remains fraught with difficulties. Social media sarcasm is often expressed in courteous language, making it challenging to detect[19]. Furthermore, the lack of large textual datasets poses a significant obstacle, as deep learning models require substantial data for effective learning.

Typically, datasets are divided into training and testing sets in learning-based methods. During training, models learn to associate specific input text with corresponding outputs, allowing them to understand documents. The testing dataset is used for prediction, with feature extractors translating concealed textual inputs into feature vectors, which are then used to generate prediction tags[20]. Hybrid approaches combining lexicon-based and machine learning methods have shown success in structured domains, but the vast volume and variability of web-based data necessitate more sophisticated techniques.

This research focuses on developing an automated algorithm to detect sarcasm on social media, particularly on Twitter. Launched in 2006, Twitter rapidly became a leading public social networking platform, with millions of active users and billions of tweets daily. Twitter users often express opinions and share news, making sarcasm detection crucial for text mining applications such as marketing, sentiment analysis, and opinion mining. After feature selection methods are applied, data will be analyzed using neural networks and machine learning algorithms for sarcasm classification, evaluated using various metrics.

The primary objectives of this research are as follows:

1. Develop a deep learning model with fine-grained parameters to effectively analyze sentiment data for sarcasm detection.
2. Incorporate Normalized Google Distance (NGD) into traditional deep learning processes to explore data beyond the scope of models like BERT.
3. Optimize classifier parameters using the Enhanced Sinogramic Red Deer (ESRD) optimizer before employing the AM-BLSTM-GRU for classification.
4. Create a flexible scheme that can be adapted for social media review analysis across different domains with minimal adjustments.

Motivation and contributions

The primary gaps identified in current sarcasm detection methods are as follows:

- Limited ability to capture complex linguistic patterns: Traditional sentiment analysis methods, particularly lexicon-based and rule-based approaches, often fail to detect nuanced sarcasm due to the inherent contradictions in sarcastic statements. Existing deep learning models such as LSTM and GRU have limitations in fully capturing these complex patterns without contextual cues.

  Our model incorporates an attention-based mechanism within the AM-BLSTM-GRU architecture, allowing the network to focus on the most important contextual information in the text. This enhances the ability to detect subtle features indicative of sarcasm. By using the attention mechanism, the model prioritizes key linguistic elements (e.g., elongated words, specific punctuation) that traditional models tend to miss, significantly improving sarcasm detection accuracy.

- Challenges in multimodal sarcasm detection: Many previous models focus solely on textual sarcasm detection, ignoring multimodal elements like emojis or images that often accompany sarcastic posts. Multimodal sarcasm detection remains underexplored, and existing methods face difficulties in integrating different data types.

  While our primary focus is on textual sarcasm detection, our model can be extended to incorporate multimodal data. By using embedded CNN layers, the architecture is adaptable for feature extraction

from both text and visual data (e.g., emojis). This flexibility positions the model for future work that includes multimodal sentiment analysis, which is critical for accurately detecting sarcasm in complex, real-world social media posts.

- Handling data imbalance and noise: Many sarcasm detection datasets suffer from class imbalance, where non-sarcastic posts vastly outnumber sarcastic ones. Additionally, external data sources, such as Google search results (used for NGD embedding), can introduce noise into the model, reducing the accuracy of sentiment analysis.

  The integration of the Enhanced Sinogramic Red Deer (ESRD) optimizer plays a pivotal role in addressing both data imbalance and noise. By fine-tuning classifier parameters and optimizing feature selection, the ESRD optimizer ensures that the model focuses on the most relevant features while minimizing the impact of noisy or irrelevant data. This process improves classification accuracy, especially when working with imbalanced datasets.

- High computational complexity and long training times: Bidirectional models like BiLSTM and BiGRU are known for their high computational cost, especially when applied to large datasets. This often results in longer training times and increased resource demands.

  While our model utilizes BiGRU for its enhanced ability to capture bidirectional dependencies, we mitigate its computational complexity by implementing adaptive learning rates and dropout regularization. These techniques help to stabilize the training process, reduce overfitting, and ensure that the model can efficiently process large datasets without excessive resource consumption.

The remainder of this paper is organized as follows: Sect. 2 reviews relevant literature, Sect. 3 presents the model briefly, Sect. 5 offers the results and discussion, and Sect. 6 concludes the study.

Related work

The attention-based transformer model presented by Sukhavasi and Dondeti[21] shows good performance in analyzing both text and emoji data. However, the accuracy rate of using only the transformer model can be lower. To address this, input data is pre-processed using techniques such as stop word removal, case folding, filtering, lemmatization, stemming, and tokenization. After pre-processing, textual characteristics are extracted using the Average-based Term Frequency-Inverse Document Frequency (TF-IDF) technique. This text model is built using the Gated Temporal Bidirectional Convolution Network (GT-BiCNet). The emoji model uses the emoji-to-vector model (E-VM), which represents characteristics as vectors. A deep feature fusion strategy combines the generated models to produce TexMoJ features. These feature vectors are classified using the Attention LSTM deep learning model, built on ALABerT, an improved bidirectional encoder representation. The Enhanced Pelican Optimisation Algorithm (EpoA) is used to reduce network model losses. The softmax layer effectively classifies the data as sarcastic or not. The proposed strategy outperforms several existing methods in various performance metrics. The English Twitter dataset achieved 99.1% accuracy, 99.2% precision, 99.1% recall, and 99.1% F-measure. The Hindi dataset achieved 98.1% accuracy, 98.41% precision, 98.2% recall, and 69.6% F-measure. The execution time for the English dataset was 56.66 s, with an average threshold of 12364.365 s.

Liu et al.[22] introduced a hierarchical fusion model for improved multimodal sarcasm detection. This model combines sentiment information with attribute-object matching in the picture modality, considered an auxiliary modality. Sentiment data from each modality is merged to create a more complete representation. A cross-modal Transformer describes inter-modal incongruity connections, and a sentiment-aware image-text contrastive loss technique brings semantics of pictures and text closer. This approach enhances comprehension of mismatched relationships and outperforms the state-of-the-art in the multimodal sarcasm detection challenge.

Prashanth et al.[23] presented a method called "Sarcasm-based Tweet-Level Stress Detection" (STSD) to identify tweet-level stress using sarcastic information. This involves modifying the logistic loss function to minimize it for non-sarcastic tweets. Dimensionality reduction is performed using kernel principal component analysis (PCA) for better performance. The STSD model significantly outperforms baseline models, with accuracy improvements between 5.25% and 9.19%. The F1-score increases by at least 0.085 points and up to 0.164 points compared to baseline representations.

Ladoja et al.[24] developed a model for detecting sarcasm in Pidgin tweets. The study evaluated Artificial Neural Network (ANN) classifiers like Vanilla, XGBoost, Random Forest, and Logistic Regression on sarcastic data from Nigerian Pidgin tweets. Evaluation metrics included accuracy, precision, recall, and F1-score. The XGBoost model achieved an accuracy of 85.78%, precision of 88.57%, recall of 94.44%, and F1-score of 91.41%, showcasing its performance. This research highlights the complexities of language in the Nigerian context.

Gedela et al.[25] proposed a new ensemble method for sarcasm detection using deep learning techniques. Contextual word embeddings created using Bidirectional Encoder Transformers are used in an ensemble of models, including Convolutional Memory and others. Classification experiments with machine learning classifiers like Random Forest, Support Vector Machine, Multinomial Naive Bayes, and Least Squares Support Vector Machine were conducted. The approach achieved an F1-score of 80.49% on the self-annotated Reddit corpus and an accuracy of 94.89% on the news headlines repository, improving by 2.99% compared to previous methods.

Lora et al.[26] introduced 'Ben-Sarc,' a large-scale Bengali corpus with 25,636 comments from various public Facebook pages, evaluated by external reviewers to study sarcasm detection. The research explores sarcasm detection using deep learning, classical machine learning, and transfer learning models. The best accuracy of 75.05% was achieved with transfer learning using Indic-Transformers Bengali Bidirectional Encoder

Representations from Transformers. Long Short-Term Memory (LSTM) achieved 72.48% accuracy, and Multinomial Naive Bayes reached 72.36%. The goal of releasing the Ben-Sarc corpus is to advance the Bengali NLP community.

Aleryani et al.[27] investigated the impact of a basic preprocessing step on sarcasm detection in Arabic. The study examined how removing emojis from datasets affects accuracy, given Arabic's rich vocabulary. Using modified AraBERT models for sarcasm detection, the research shows that removing emojis can improve accuracy by focusing on language comprehension. This approach efficiently navigates the complexities of Arabic sarcasm, providing insights for social media platforms and setting new standards for Arabic natural language processing.

Liu et al.[28] studied sarcasm detection on the social Internet of Things, considering model parameters and inter-modal interactions. They proposed lightweight multimodal interaction models with deep learning-based knowledge enhancement, integrating visual commonsense algorithms to improve semantic information of picture and text modal representation. A multi-view interaction technique facilitates connections between modal perspectives. The model outperforms unimodal baselines and performs similarly to multimodal baselines with fewer parameters.

Bousmaha et al.[29] developed a thorough approach for sarcasm detection in the Algerian dialect, including text and visual analysis. They combined linguistic features with machine learning for text analysis and used the VGG-19 model for image classification and EasyOCR for Arabic text extraction. The goal is to develop a robust system for detecting sarcasm in visual and textual content in the Algerian dialect, achieving 89.28% accuracy for the visual model and 92.79% for the textual model.

Galal et al.[30] constructed an Arabic sarcastic corpus and fine-tuned three pre-trained Arabic transformer-based language models for sarcasm detection. They proposed a mixed deep learning method combining static and contextualized representations with pre-trained language models like BERT for Arabic resources. The hybrid method achieved an F1-score improvement of 5% on a benchmark dataset, outperforming state-of-the-art models by 8%.

Shiwakoti et al.[31] explored the complex world of climate change discourse on Twitter using the ClimaConvo dataset of 15,309 tweets. Their annotations cover topics like humor, position, hate speech, and relevance, offering a detailed understanding of discourse dynamics. The study benchmarks algorithms for six tasks, including humor analysis and hate speech identification, revealing trends in hate speech, stance prevalence, and tweet distribution. Advanced topic modeling uncovers underlying subject clusters, providing insights for researchers, politicians, and journalists.

Meng et al.[32] proposed an attention mechanism and pre-training model for sarcasm detection, focusing on context to extract semantic features from phrase fragments. An intra-sentence attention mechanism models semantic features by weighting important phrases. The method outperforms baselines and state-of-the-art models in experiments on a public dataset.

Ahire et al.[33] suggested that detecting sarcasm on Twitter requires considering both tweet content and general user behavior. Their method analyzes user behavior and tweet context, examining how actions impact others and how users act when detecting sarcasm. The approach uses contextual data and behavior patterns to implement a general method for sarcasm detection on Twitter.

Rajani et al.[34] proposed a method for improving sentiment analysis by reliably detecting sarcasm. The study focuses on lexical, sarcastic, and contextual aspects, using feature sets to classify tweets as sarcastic or not. A sarcastic feature set combined with a hybrid machine learning strategy improves accuracy. The hybrid method achieves a 97.3% correctness rate for sarcastic feature sets, outperforming other machine learning methods.

Sahu and Hudnurkar[35] introduced an approach for sarcasm detection that classifies words into sarcastic and non-sarcastic categories. Pre-processing involves stop word tokenization, followed by extracting features based on information gain, chi-square, and symmetrical uncertainty. A hybrid optimization technique called Clan Efficient Grey Wolf Optimization (CU-GWO) is used for optimal feature selection and optimizes DCNN weights for sarcasm detection. The proposed algorithm's efficacy is compared with existing methods using various metrics.

The Comprehensive Comparative Analysis of Existing Sarcasm Detection Methods is presented in Table 1.

Proposed methodology for sentiment analysis with sarcasm detection

Sarcasm is the use of statements with an obviously negative connotation intended to amuse, irritate, or ridicule someone. Traditional sentiment analysis approaches struggle to identify sarcasm, leading to less accurate results when sarcasm is present in a document. This study proposes a methodology for sentiment analysis that incorporates sarcasm detection. By using pre-processed data, the system trains a deep learning network to perform sentiment and sarcasm classifications simultaneously. Evidence suggests that multi-task learning can enhance the effectiveness of learning and improve the accuracy of predictions made by task-specific models. Figure 1 illustrates the proposed framework's design, and the remainder of this section provides a detailed explanation of each component within the framework.

Data acquisition

Collecting data during the data acquisition phase of machine learning and deep learning often requires significant time and energy, especially when determining the relevance of data. For the proposed approach, both a sentiment dataset and a sarcasm dataset are necessary. The dataset used in this work was acquired from Kaggle, consisting of 227,599 tweets labeled as "neutral," "negative," or "positive" for sentiment analysis[36].

The links to these datasets are provided below:

- Kaggle twitter sentiment dataset: https://www.kaggle.com/datasets/cosmos98/twitter-and-reddit-sentimental-analysis-dataset.

Authors (Year)	Model/Approach	Dataset	Accuracy / F1-Score	Key Limitations	Key Strengths
Sukhavasi and Dondeti (2024)[21]	Transformer (ALABerT) + Attention LSTM	English, Hindi Twitter data	Accuracy: 99.1% (English)	Limited emoji integration; High training time	Multilingual sarcasm detection
Liu et al. (2024)[22]	Sentiment-aware Hierarchical Fusion with Cross-modal Transformer	Multimodal Sarcasm Challenge	Not specified	Focused mainly on multimodal input; Complex fusion process	Cross-modal sentiment learning
Prashanth et al. (2024)[23]	Sarcasm-based Tweet-Level Stress Detection (STSD)	Twitter	Accuracy improved 5.25-9.19%	Focused on stress detection rather than sarcasm alone	Sarcasm-enhanced stress detection
Ladoja and Afape (2024)[24]	XGBoost, ANN on Pidgin Tweets	Nigerian Pidgin corpus	Accuracy: 85.78%	Language-specific; Limited scalability	Focus on low-resource languages
Gedela et al. (2024)[25]	Deep Contextual Ensemble (Contextual Word Embeddings)	Reddit, News Headlines	F1: 80.49% (Reddit), 94.89% (News)	Lower performance on noisy data	Ensemble with contextual features
Lora et al. (2024)[26]	Transfer Learning (Indic-Transformer for Bengali)	Ben-Sarc (Bengali Facebook)	Accuracy: 75.05%	Focused on a single low-resource language	Resource release for Bengali NLP
Aleryani et al. (2024)[27]	Modified AraBERT (emoji exclusion study)	Arabic sarcasm dataset	Improved performance after emoji removal	Sensitive to emoji preprocessing	Impact of emoji handling
Liu et al. (2024)[28]	Lightweight Multimodal Interaction Model with Visual Commonsense	Social IoT platforms	Comparable to multimodal baselines	Focused on IoT; Limited to multimodal	Lightweight model with reduced parameters
Bousmaha et al. (2024)[29]	VGG-19 for image + Textual Machine Learning	Algerian dialect tweets and images	Text accuracy: 92.79%, Visual: 89.28%	Dialect-specific; multimodal complexity	Combines text and visual sarcasm
Galal et al. (2024)[30]	Hybrid Deep Learning (Arabic BERT + Static embeddings)	Arabic sarcastic corpus	F1 improved by 5%	Language-specific models	Fine-tuning Arabic transformers
Shiwakoti et al. (2024)[31]	Multi-aspect Analysis on Climate Change Twitter Data	ClimaConvo dataset (15,309 tweets)	Not directly specified	Focus on broader discourse (stance, humor)	Comprehensive social discourse annotation
Meng et al. (2024)[32]	Attention Mechanism + Pre-training model	Public sarcasm dataset	Outperformed existing baselines	Focused mainly on intra-sentence sarcasm	Phrase-level sarcasm modeling
Ahire et al.[33]	Contextual User Behavior + Tweet Content Analysis	Twitter	Not specified	Requires user profile information	Behavior-based sarcasm detection
Rajani et al. (2024)[34]	Hybrid Machine Learning with Feature Sets	Twitter	Accuracy: 97.3%	Limited in complex sarcastic relations	Sarcastic feature set design
Sahu and Hudnurkar (2024)[35]	Metaheuristic-assisted DCNN (Clan Efficient Grey Wolf Optimization)	Social media sarcasm data	Not explicitly mentioned	Computationally intensive optimization	Feature selection + deep ensemble

Table 1. Comprehensive comparative analysis of existing sarcasm detection Methods.

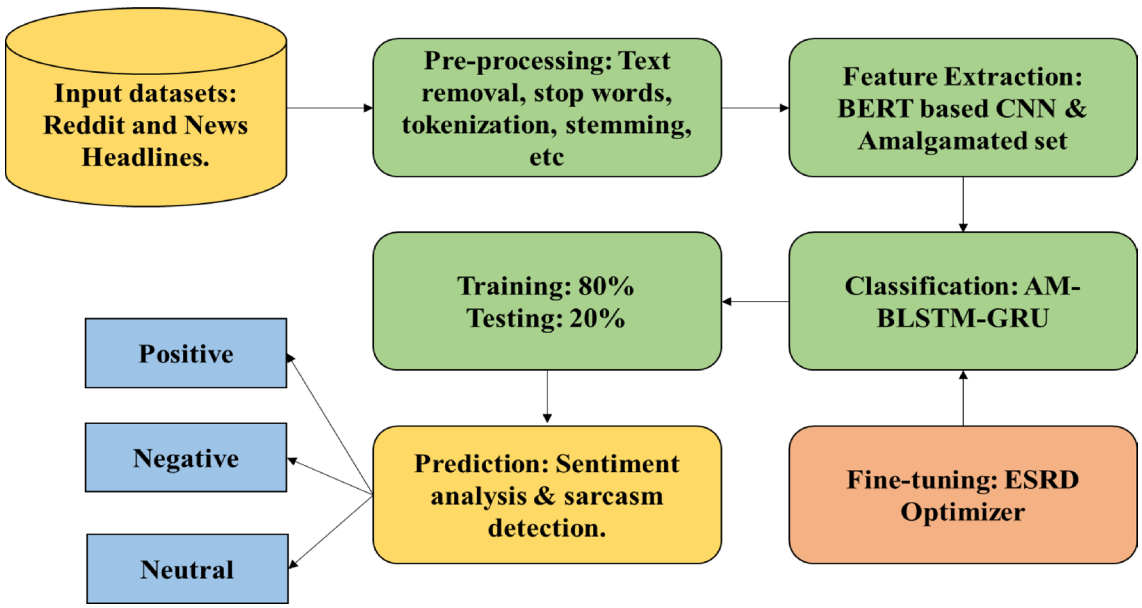


Fig. 1. Workflow of the research work.

- Kaggle news headlines sarcasm dataset: https://www.kaggle.com/datasets/rmisra/news-headlines-dataset-for-sarcasm-detection.

	clean_comment	label
0	I`d have responded, if I were going	neutral
1	Wow... u just became cooler.	positive
2	Sooo SAD	negative
3	bullying me	negative
4	leave me alone	negative

Fig. 2. Sample of first dataset.

	is_sarcastic	headline
0	1	thirtysomething scientists unveil doomsday clo...
1	0	dem rep. totally nails why congress is falling...
2	0	eat your veggies: 9 deliciously different recipes
3	1	inclement weather prevents liar from getting t...
4	1	mother comes pretty close to using word 'strea...

Fig. 3. Sample of second dataset.

Figure 2 presents example headers to provide a better understanding of the dataset. There is a slight imbalance, with the "neutral" label being predominant and "negative" as the minority class. The sarcasm dataset comprises 28,619 tweets marked as sarcastic and none as non-sarcastic. Figure 3 illustrates examples of both sarcastic and non-sarcastic tweet headers[37– 38]. Similarly, there is a slight imbalance in the sarcasm dataset due to the higher number of non-sarcastic tweets (type 0).

Data pre-processing

To ensure data quality and enhance sarcasm detection accuracy, the following preprocessing steps were applied:

- Lowercasing text: Converts all text to lowercase to maintain consistency and remove case sensitivity.
- Stopword removal: Eliminates common stopwords (e.g., *the*, *is*, *and*, *but*) that do not contribute to sentiment understanding.
- Punctuation removal: Removes unnecessary punctuation marks (e.g., commas, exclamation marks) to streamline text representation.
- Stemming and lemmatization: Reduces words to their root forms (e.g., *running* → *run*, *caring* → *care*) for uniformity in feature extraction.
- Tokenization: Splits text into individual words or subwords for better text representation in deep learning models.
- Handling emojis: Emojis play a critical role in sarcasm detection as they often convey implicit sentiment. For instance, a phrase like "Oh great!" has a sarcastic intent that may be lost if emojis are removed. To retain this information:

  We use emoji-to-text mapping, where each emoji is converted into its sentiment-bearing equivalent (e.g., → happy, → annoyed).

  Pre-trained Emojinet embeddings are integrated to help the model recognize emojis as contextual cues for sarcasm.

- Processing hashtags: Hashtags often encapsulate sarcasm through wordplay or exaggeration (e.g., *#BestDayEver* in a negative context). To retain their significance:

  Multi-word hashtags are split into individual words (e.g., *#NotHappy* → *Not Happy*), ensuring better sentiment analysis.

  Hashtags that provide sentiment cues (e.g., *#Sarcasm*, *#Ironic*) are retained as additional **contextual features** to improve sarcasm classification.

- Handling User Mentions: Removes or anonymizes user mentions (@username) to prevent model bias and focus on textual sarcasm patterns.

These preprocessing steps improve the dataset quality by reducing noise and enhancing feature representation, ensuring optimal sarcasm detection performance.

Feature extraction models

Rationale for component selection

Normalized google distance (NGD) NGD was selected for its ability to quantify the semantic similarity between words based on their co-occurrence in search engine results. In sarcasm detection, where subtle contextual cues can drastically change the meaning of words, NGD provides a means to capture hidden relationships between sarcastic expressions and their intended meanings.

Unlike traditional embedding methods (Word2Vec, GloVe, FastText), NGD leverages search engine frequency data to dynamically assess word relationships. This ensures that sarcastic expressions, which often involve unexpected word associations, can be detected more effectively. For example, a phrase like *"Oh great, another Monday!"* has a sarcastic meaning that traditional embeddings may fail to capture, whereas NGD can infer sarcastic intent by analyzing web-based co-occurrences.

Bidirectional encoder representations from Transformers (BERT) BERT was chosen because of its state-of-the-art performance in capturing contextual word meanings through pre-trained deep bidirectional transformers. It excels at understanding word sequences and the relationships between them, which is crucial in sarcasm detection, where the tone of a statement often contradicts its literal meaning.

BERT enhances the model's ability to grasp context-sensitive information, ensuring that the sarcastic intent of a statement is identified based on both the surrounding text and the overall context.

Enhanced sinogramic red deer (ESRD) optimizer The ESRD optimizer was chosen due to its capability to address data imbalance and optimize classifier parameters for sarcasm detection[39]. Traditional optimizers such as Adam and RMSprop focus on gradient-based adjustments, which can sometimes lead to local optima when dealing with highly imbalanced datasets.

ESRD introduces a metaheuristic approach inspired by the hierarchical social behavior of red deer. The optimizer improves feature selection by iteratively refining classification parameters, ensuring that noisy or irrelevant features are eliminated. This is particularly beneficial for sarcasm detection, where irrelevant features (e.g., generic words without sentiment weight) can reduce model performance.

Additionally, ESRD maintains an optimal balance between exploration and exploitation during optimization, allowing the model to dynamically adjust weight parameters and prevent overfitting to non-sarcastic patterns. The use of ESRD leads to a more robust and efficient sarcasm classification model, especially in cases where sarcastic expressions are subtle and complex.

Proposed embedded CNN (NGD + BERT)

NGD and BERT are integrated to enhance feature extraction and contextual understanding in sarcasm detection. BERT generates contextualized word embeddings by capturing bidirectional dependencies in text, effectively handling ambiguous phrases. However, BERT alone struggles with sarcasm due to its reliance on pre-trained corpora, which may not fully capture real-world semantic relationships.

To address this limitation, NGD quantifies semantic distance between words using web search frequencies. This allows the model to detect sarcastic expressions where literal meanings contradict intended sentiment. For example, in the phrase *"Oh great, another Monday!"*, NGD identifies that *"great"* and *"Monday"* frequently co-occur in sarcastic contexts, refining the BERT-based representation for improved classification.

Traditionally, deep learning models have used Word2Vec, GloVe, and FastText for feature representation. However, these models rely on fixed pre-trained embeddings and lack shared representations, making them ineffective for handling Out-Of-Vocabulary (OOV) terms. To overcome this, BERT was introduced with WordPiece Tokenization, enabling it to manage OOV issues more effectively[40–41]. Despite this advancement, BERT embeddings still have limitations:

- The semantic importance of words diminishes if they appear as substrings within other words.
- Lack of supplementary contextual details makes it difficult to differentiate relationships between sarcastic and non-sarcastic text.
- Uncommon words remain difficult for BERT to interpret accurately.

To enhance sarcasm detection, our study integrates NGD and BERT within a CNN-based feature extraction model. The framework consists of:

- A single embedding layer, where input text is processed.
- Two parallel sub-layers (NGD and BERT), enriching the semantic context.
- Four convolutional neural network (CNN) layers, refining extracted features.
- A fully connected layer and softmax classifier, categorizing input text as sarcastic or non-sarcastic.

This integration effectively leverages NGD for external semantic understanding while utilizing BERT for contextual representation, significantly improving sarcasm detection accuracy.

Aspects can be phrases with beginning and ending letters (B and I), while non-aspect words, abbreviated as "Opinion," are represented by the letter O. For the sake of clarity, assume that $x = \{x_1, x_2, x_3, \ldots, x_n$ as a

sequence of word as w_{bert} and w_{ngd}. The first embedding based on BERT w_{bert} stands for embeddings that have been trained using a massive collection of general-purpose text, typically consisting of billions of tokens. Using NGD (Normalized Google Distance), which measures semantic similarity based on word co-occurrence in web search results, the second embedding, w_{ngd}, is generated to refine feature representations. That is why the Normalised Google Distance (NGD)[42] was developed for that specifically. The number of results refunded by Google for a specific set of keywords is used to construct NGD, a semantic similarity metric.

Regarding the convolutional neural network (CNN) layers, it has a plethora of filters, each of which has a fixed kernel size of $k = 2c + 1$. Each specific position $i = 1, \ldots, n$. Every filter plays a role in creating the i th word, which contains 2c neighbouring words. The final vector set was obtained by applying a fully linked layer in conjunction with the softmax layer.

In the case of combined feature set $AF = \{af_1, af_2, \ldots, af_n\}$ is given as input to NGD besides BERT in the procedure of Eq. (1).

$$EMD_{CNN} = \left\{ngd_{af_1} \left| betr_{af_1}, ngd_{af_2} \right| betr_{af_2} \ldots\ ngd_{af_n} | betr_{af_n} \right\} \tag{1}$$

Equation (2) makes it quite evident that NGD gave each combined set of features an association-based score.

$$NGD\left(af_x, af_y\right) = \frac{high\left\{log_2 f\left(af_x\right), log_2 f\left(af_y\right)\right\} - log_2(af_x, af_2)}{log_2 T - low\left\{log_2 f\left(af_x\right), log_2 f\left(af_y\right)\right\}} \tag{2}$$

On Bert input illustration fed into L_s blocks.

$$hid_i^l = Trans\left(hid_i^{l-1}\right),\ Where\, l_i = l_1, l_2, l_3 \ldots\ L_s \tag{3}$$

The transformer state as $\left\{hid_i^{Ls}\right\}(i = 1^N) \in R(dim_1)$. Achieving a pooled sequence using the first token < CLS > as hid_0^{Ls} A one-of-a-kind, top-notch, and flawless word-to-vector presentation has been created by utilising a post pad sequence with BERT and NGD. The next step was to convert the words to vectors using Eq. (4) and the multiplicative law of statistics. There are exactly m possible results in the first experiment of a compound experiment, while there are n possible outcomes in the second experiment for every result in the first. In that case, there are exactly m x n results from the experiment.

$$E_{i\times j} = m \times\ n \tag{4}$$

Amalgamated feature set

Most reviews are biased reflections of the reviewer's positive or negative views, expressed through various perspectives. Syntactic structure, language patterns, and dependency relations have been employed in many studies to analyze these perspectives. Although these methods are impressive, the complexity of reviews has often prevented them from achieving optimal results. To create a comprehensive collection of features, this study used a structure-based natural language processing model. This model checks the dependency of specific opinions on review aspects through dependency parsing. The goal is to enhance dependency-based techniques to identify the best patterns for finding aspect and opinion terms. To address bidirectional issues, a "forward/reverse" programming technique is used with preset tags like "#neg". Most words have been adjusted to handle multiple nouns. For example, in the sentence "food good and delicious but the staff was not good," there are several nouns. We then move on to the classification phase with the acquired feature set, which includes items like [R1? (Poor, Colours {0}), (Great, Battery {1})].

Classification using AM-BLSTM-GRU model

Attention based BI-LSTM GRU

The RNN uses static-length vectors to demarcate data orders. A certain moment represents each unique component[43]. Additionally, the outcome is affected for a certain moment by a set of data from instant *t*− 1 which is characterized in (5) and (6):

$$h(t)\ = f\left(X(t) \times\ U + h(t\ -\ 1) \times\ W + b\right) \tag{5}$$

$$o(t) = g\left(h(t) \times\ V + c\right) \tag{6}$$

Where, the activation functions signified as *U*, *W*, suggest *c* network which is represented as *f* and *g*. While conventional recurrent neural networks can process data pertaining to short-term structures, they are unable to process data pertaining to long-term sequences. A layer of gates covers the structure of LSTM. Input data is fed into the LSTM, which then supplies it to the gate at the output. The computational complexity is a problem for the Bi-LSTM architecture, though. () due to Bi-LSTM. The illustration of layer construction of Bi-LSTM is presented in Table 2.

With the use of bidirectional instant in a dated of b, an ideal dependence is achieved. Therefore, when paired with a (GRU), LSTM is the superior choice for achieving endurance. The two doors that make up the GRU are the updated door (z_t) and the reset door (r_t). To regulate the amount of information transfer from one state to another, the reset gate is used, and the update gate is used to supervise knowledge of the preceding state. Figure 4 shows the GRU structural diagram.

Sum of forget gates	4
Activation function	Sigmoid, tanh
Sum of input gates	4
Sum of output gates	4

Table 2. Illustration of layer construction of Bi-LSTM.

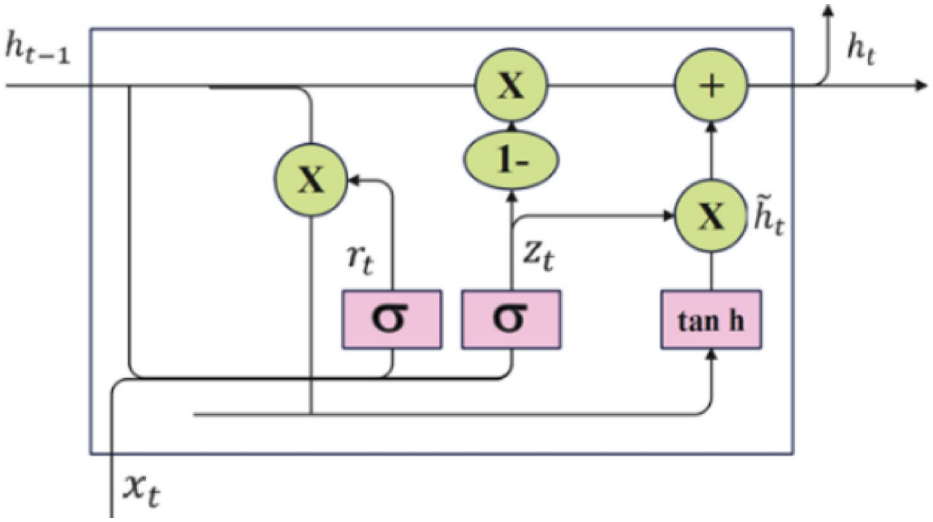


Fig. 4. Structural diagram of GRU.

However, GRU alone cannot deliver a respectable level of accuracy for sarcasm prediction. This study presents a hybrid of Bi-LSTM and GRU apparatus to address the shortcomings of both methods. Formulas for GRU are provided in (7) through (11).

$$r\,(t) \;=\; \sigma\;(W(\,r).\,[h(t\;-\;1),\;x\,(t)\,]) \tag{7}$$

$$z\,(t) \;=\; \sigma\;(W(\,z).\,[h(t\;-\;1),\;x\,(t)\,]) \tag{8}$$

$$\bar{h}\,(t) \;=\; tanh(W\bar{h}.\,[r1\;\times\;\;h(t\;-\;1),\;x\,(t)\,]) \tag{9}$$

$$h\,(t) \;=\; (1\;-\;z(t)\;\times\;\;h(t\;-\;1)\;+\;z(t)\;\times\;\;\bar{h}(t) \tag{10}$$

$$y\,(t) \;=\; \sigma\;(W(\,o).\,h(t\,)) \tag{11}$$

Bi-LSTM GRU with an attention mechanism

Predicting sarcasm detection is achieved by combining features from Bi-LSTM GRU with an attention technique. The Bi-LSTM network's sequential nodes have their hidden features extracted, and those nodes are then used to further explain the sequence. By including the attention layer into the Bi-LSTM layer, the impact of important nodes is enhanced. Critical nodes are clustered using the attention mechanism, which incorporates the text content phenomenon. To represent the critical node carrying vital information inside the input sequence, a sequence vector is also constructed. Sequences are represented using a combination of Bi-LSTM GRU mechanism. Represent the attention apparatus as a whole mathematically in Eqs. (12)–(14).

$$u_{it} = \tanh(W_n h_{it} + b_n) \tag{12}$$

$$a_{it} = \frac{exp\left(u_{it}^T u_n\right)}{\sum_t exp\left(u_{it}^T u_n\right)} \tag{13}$$

$$S_i = \sum\nolimits_t a_{it} h_{it} \tag{14}$$

Throughout the training phase of the model, a contextual vector with correlated weights is used to assess the input at the node level. The dot product similarity is used to construct the node level vector, which is then supplied into the softmax layer. The weighted vector sequence receives its input from the softmax layer. Just like the one-layer MLP evaluates the sequential weight of hidden vectors, the dot product similarity's hidden vectors

are input into it. Dot to estimate the weighted sum of nodes. The linear layer receives the important information, and the sum of hidden nodes in the Bi-LSTM layer is proportional to that.

Fine-tuning using an enhanced sinogramic red deer

Minimising the feature dimensionality without sacrificing performance is the primary goal of the tuning process. Use of Radon transformation to translate the proposed classifier's feature maps into sinograms is now underway. The next step is to change each sinogram to y^{s} of size $\left(1 \times \frac{mnc}{4}\right)$, so for N samples, we have pool y^{s} of size $\left(N \times \frac{mnc}{4}\right)$.

The conventional red deer algorithm The RD algorithm is an innovative optimisation method that draws inspiration from nature. It is an algorithm that falls under the category of metaheuristics. The key benefit of the RD method is that it keeps the exploration and exploitation phases equal, which aids in the low-complexity assignment of important features.

Male and female red deer can be found (hinds). A herd of hinds is known as a harem. A male commander is appointed to each harem. Male RDs will roar and battle in an effort to increase the number of hinds in their harem. Male RDs are classified as either stags or commanders based on the intensity of their roaring phase. The harem's leader will be chosen from among the males only after a tough war. Here, the $\left(\frac{mnc}{4}\right)^{\text{th}}$ encoded 1D features in $Y^{S} = \left[\mathcal{Y}_1, \mathcal{Y}_2, \ldots ., \frac{\mathcal{Y}_{mnc}}{4}\right]$, where $\mathcal{Y}_i$ RDs are grouped by feature vectors of size (N, 1). When solving an optimisation problem, finding a solution that is close to optimal, taking all relevant variables into account, is the primary goal.

Stage 1: Initialize the population.

At this phase, an $\mathcal{N}_r$ sparse features to prime red deers as, $G = [\mathcal{Y}_1, \mathcal{Y}_2, .., \mathcal{Y}_{\mathcal{N}_p}]$.Among this population, $\mathcal{N}_{male}$ are chosen as features, G_{male},while the rest are $\mathcal{N}_{hind} = \mathcal{N}_p - \mathcal{N}_{male}$ the hind group, G_{hind}.Based on their fitness value, the top topographies in the chosen population are chosen as males. The proposed goal or function is a weighted combination of the amount of the chosen number of red deer and the classification accuracy, which is based on the LSTM classifier.

$$f = \omega \, .acc + (1 - \omega \,) \frac{\vartheta}{\Gamma} \tag{15}$$

As long as the red deer or traits that are currently being used for classification have an accuracy of at least acc. The sum of red deers that are now being picked is denoted by ϒ, while the total sum that is being targeted is represented by Γ. The range of values for the weighting coefficient ϋ ranges from 0 to 1.

Stage 2: Roaring phase

As things stand, the best option. Searching for other nearby outstanding attributes is what roaring is all about. The procedure for updates is this:

$$RD_{male}^{new} = \begin{cases} RD_{male}^{old} + a_1 \times \left(\mu \; - \not{\ell} * a_2 + l, \;\; if \; a_3 \geq \; 0.5 \right. \\ RD_{male}^{old} - a_1 \times \left(\mu \; - \not{\ell} * a_2 + l \quad if \; a_3 < 0.5 \right. \end{cases} \tag{16}$$

where RD_{male}^{new} male and RD_{male}^{old} male is the deer solutions. μ and ℓ are search of neighboring solutions. α1, α2, and α3 are coefficients that are created at random from a uniform distribution with values between zero and one. The next step is to divide two groups: stage and commander.

The number of male commanders, $\mathcal{N}_{com,}$ is measured as $\mathcal{N}_{com,} = round\left(\delta \, \mathcal{N}_{male}\right)$,where δ is a random range from 0 to 1. The sum of stags can be uttered $\mathcal{N}_{stag} = \mathcal{N}_{male} - \mathcal{N}_{com}$

Stage 3: The fighting phase between stags besides commanders.

Here, commanders are free to combat with stags of their choosing. When it comes to the group solution of commanders G_{com}, we allow it to approximate that of stags G_{stag}.Accordingly, two solutions are produced as

$$G_{new_1} = \frac{G_{com} + G_{stag}}{2} + \beta_{\,1} \times \left((u - \not{\ell} * \beta_{\,2} + \not{\ell}\right. \tag{17}$$

$$G_{new_2} = \frac{G_{com} + G_{stag}}{2} + \beta_{\,1} \times \left((u - \not{\ell} * \beta_{\,2} + \not{\ell}\right. \tag{18}$$

where G_{new_1} and G_{new_2} denote the new generated process. u and ℓ are search space. $\beta_{\,1}$ and $\beta_{\,2}$ are coefficients that are created with values between zero and one. Four options are available to us now., i.e., G_{com}, G_{stag}, G_{new_1} and G_{new_2}, the solution commander.

Stage 4: Forming harems.

Harem formation is the responsibility of the newly appointed commander in this case. One male deer (the hind) and several female deer (the harem) make up a herd. The commander's skills in roaring and battling determine the random distribution of the hinds into various harem i, $\mathcal{N}_{hind}^{harem^{i}}$, is calculated as

$$\mathcal{N}_{hind}^{harem^{i}} = round(f_i, \mathcal{N}_{hind}) \tag{19}$$

where f_i signifies power, fitting charge, of the chief.

Stage 5: Mating phase.

Once harems have been established, three distinct mating strategies can be employed. There are two scenarios where the leader of a harem can mate: the first is when the leader mates with Ↄ of the hinds from their own harem, besides the second is when the leader mates with ϑ of the hinds from other harems. To extend his control area, commanders launch an assault on another harem. Thirdly, harem limitations don't stop stags from mating with the closest hind. Due offspring RDs, G_{OS}, i.e., keys, are produced as

$$G_{\mathrm{OS}} = \frac{G_{com} + G_{hind}}{2} + \theta \times (u - \not{\mathcal{J}} \tag{20}$$

where G_{com}, and G_{hind} are the group of keys that represent hinds. θ is a random number among 0 besides 1. In likelihood of mating G_{com} is swapped by the stag solutions G_{stag}.

In the end, a roulette wheel is used to assign the next generation based on the top commanders. When iterations have been achieved and the best features have been defined, the process returns to the beginning.

The proposed improved red deer optimizer (ERD) The proposed ERD is based on a tweaking method that improves the standard red deer optimizer's mating phase, as shown in Eq. 20. In the tinkering strategy, members from the harems with the worst fitting values are added to the harems with the best fitting value so far, known as the "best-so-far" harems. Depending on the level of variation among harems, this tweaking method is executed.

The tinkering strategy For the tinkering policy, we need to define the $\mathcal{N}_{harem}^{best}$ finest harems and the worst $\mathcal{N}_{harem}^{worst}$ harems, rendering to value, while keeping $\mathcal{N}_{harem}^{best} > \mathcal{N}_{harem}^{worst}$. If the original hinds are good, then the poorest of the U group's hinds will mess with the best of the Q group's harems. Nevertheless, local minima entrapment could result from this tweaking approach. Consequently, we must manage the specified quantity of the worst harems $\mathcal{N}_{harem}^{worst}$, as follows.

$$\mathcal{N}_{harem}^{worst} = \mathcal{N}_{harem}^{Total} max\left(0.5\frac{f_{harem}^{worst}}{f_{harem}^{best}}\right) - round\left(\frac{\ddot{I}}{maxiter}\left(\mathcal{N}_{harem}^{Total} - \underbrace{n}\right)\right) \tag{21}$$

Where $\mathcal{N}_{harem}^{Total}$ is the full sum of built harems. f_{harem}^{worst}, and f_{harem}^{best} represent the entire worst and best fitting charge, correspondingly. Ï current repetition sum corresponding to the maximum assigned sum of repetitions, maxiter. n is a fixed sum within each repetition.

After decisive are allocated as $\mathcal{N}_{harem}^{best} = \mathcal{N}_{harem}^{worst} + 3$. Next, the finest harems divide up the worst harems' hinds evenly among themselves, while the top harems use this distribution to simulate the impact of mutation on metaheuristic algorithms by fitting the worst hinds. The harems have been reorganised in accordance with the tweaking plan. New generations of RDs are determined by the mating phase., G_{OS},are generated as

$$G_{\mathrm{OS}} = \begin{cases} \frac{G_{com}+G_{hind}}{2} + \theta \times (u - \not{\mathcal{J}}, & if\ v > \varsigma \\ \frac{G_{com}+G_{hind}+G_{tink}}{3} + \theta \times (u - \not{\mathcal{J}}, & otherwise \end{cases} \tag{22}$$

where G_{tink} is the ξ% that the new tinkering hinds have brought forth. Here, v represents the variation in fit among the harem, which is defined as

$$v = \left(\sum_{i=1,j=1}^{\mathcal{N}_{hind}^{harem}} \left| f_{harem_i} - f_{harem_{j \neq i}} \right|^2\right)^{1/2} \tag{23}$$

where f_{harem_i} is the whole fitting standards of hinds in $harem_i$.

Results and discussion

The experimental setup and data analysis of the suggested model are labelled in depth in this section.

Experimental setup

This section examines the scientific procedures that were put in place to carry out the research experiments. Table 3 outlines the hardware and system specifications used to conduct the experiments. These include processor details, CPU clock speed, cache memory, and address size capabilities. The experiments were executed on a machine equipped with an Intel CPU running at 2.20 GHz, supporting 46-bit physical and 48-bit virtual address sizes, ensuring efficient handling of deep learning computations and large-scale datasets. The training and

Specification	Scheme parameter
Genuine Intel	Vendor_id
79	Model
Intel(R) CPU @2.20 GHz	Model name
2199.998	CPU MHz
56,320 KB	Cache size
46 bits physical, 48 bits virtual	Address sizes

Table 3. The system parameters and specifications to conduct our study experiments.

testing of all models, including the proposed sarcasm detection framework, were performed using TensorFlow 2.9.2, Keras 2.9.0, and Scikit-Learn 1.0.2 libraries.

Experimental Indicators:

The performance of the models was evaluated using the following metrics:

- Accuracy: Measures the percentage of correctly classified instances among the total number of samples.
- Precision: Indicates the proportion of correctly predicted positive observations to the total predicted positives.
- Recall (Sensitivity): Measures the proportion of actual positives correctly identified.
- F1-Score: Harmonic mean of precision and recall, providing a balance between the two metrics.

Comparison methods:

The proposed model's performance was compared against several baseline models such as XGBoost, SVM, DCNN, ALABerT, LSTM, GRU, BiLSTM, and BiGRU.

For fair comparison, the basic forms of these models were re-implemented where necessary on the same datasets (Twitter Sentiment Dataset and News Headlines Sarcasm Dataset) to maintain consistency in evaluation. Average performance was reported after multiple runs to ensure robustness.

Accuracy and loss study of the proposed classical

Figures 5 and 6 presents the validation accuracy and loss of the projected model that shows the active performance for sarcasm detection from the input datasets.

Validation analysis of proposed model on Twitter data length sequence

Table 4 presents a comparative evaluation of different sequence-based models (GRU, BiGRU, LSTM, BiLSTM) across varying input sequence lengths. These models serve as baseline techniques in the study, and their performance in terms of training time, training accuracy, and test accuracy helps establish a benchmark against which the proposed model is later validated.

In evaluating the performance of the proposed model on Twitter data across different sequence lengths, we examined the performance of various algorithms, including GRU, BiGRU, LSTM, and BiLSTM.

For a sequence length of 50, the GRU model achieved a training accuracy of 0.9711 and a test accuracy of 0.9744, with a training time of 244 s. BiGRU showed a higher training accuracy of 0.9824 but a lower test accuracy of 0.9492, requiring 382 s for training. The LSTM model recorded a training accuracy of 0.9639 and a test accuracy of 0.9582, with a training time of 229 s. BiLSTM demonstrated a significantly lower test accuracy of 0.6023 despite a training accuracy of 0.9607, taking 424 s to train.

With a sequence length of 75, the GRU model showed improved performance, achieving a training accuracy of 0.9894 and a test accuracy of 0.9834, in 251 s. The BiGRU model reached a high training accuracy of 0.9900 but had a lower test accuracy of 0.9553, with a training time of 390 s. LSTM performed well with a training accuracy of 0.9825 and a test accuracy of 0.9782, taking 317 s. BiLSTM had a training accuracy of 0.9859 and a test accuracy of 0.8644, with the longest training time of 540 s.

For the longest sequence length of 115, GRU achieved the highest test accuracy of 0.9955 and a training accuracy of 0.9927, with a training time of 339 s. BiGRU showed a training accuracy of 0.9904 and a test accuracy of 0.9854, requiring 626 s. LSTM recorded a training accuracy of 0.9853 and a test accuracy of 0.9951, with a training time of 382 s. BiLSTM had a training accuracy of 0.9843 and a test accuracy of 0.8777, with the longest training time of 646 s.

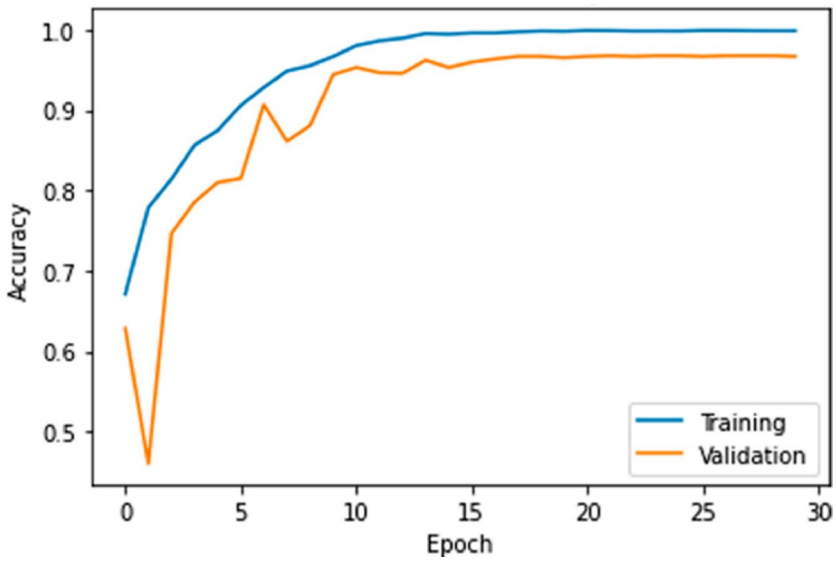


Fig. 5. Accuracy of the proposed training besides testing data.

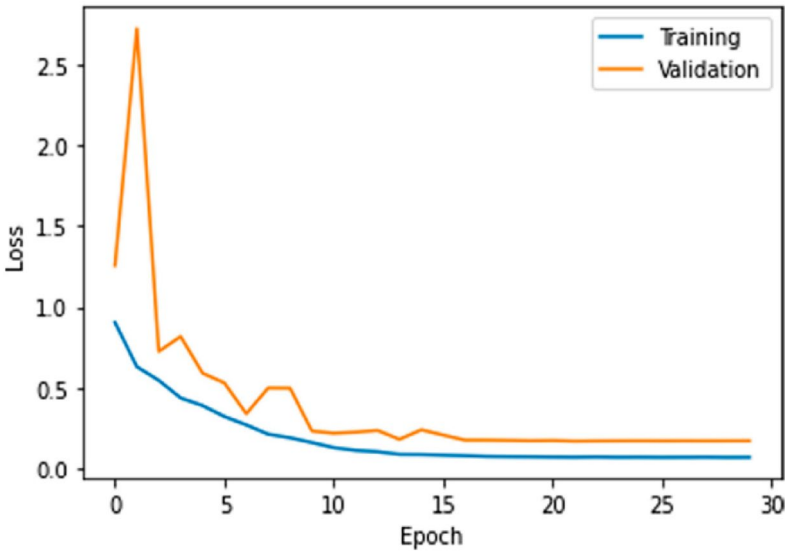


Fig. 6. Loss of the proposed-on training and testing data.

Algorithm	Training time (s)	Sequence length	Training accuracy	Test accuracy
GRU	244	50	0.9711	0.9744
BiGRU	382	50	0.9824	0.9492
LSTM	229	50	0.9639	0.9582
BiLSTM	424	50	0.9607	0.6023
GRU	251	75	0.9894	0.9834
BiGRU	390	75	0.9900	0.9553
LSTM	317	75	0.9825	0.9782
BiLSTM	540	75	0.9859	0.8644
GRU	339	115	0.9927	0.9955
BiGRU	626	115	0.9904	0.9854
LSTM	382	115	0.9853	0.9951
BiLSTM	646	115	0.9843	0.8777

Table 4. Twitter length sequence analysis of proposed model.

Batch size	Learning rate	Average accuracy
2800	0.001	99.11
2000	0.001	98.96
1000	0.001	99.00
500	0.001	98.95
100	0.001	98.93

Table 5. Analysis of proposed model on changing the batch sizes.

Overall, GRU consistently demonstrated strong performance in both training and test accuracy across all sequence lengths, followed by LSTM and BiGRU. BiLSTM exhibited more variability in test performance, indicating potential sensitivity to sequence length and longer training times.

Performance of proposed model by modifying the batch size

Table 5 explains the presentation of the projected model in terms of accuracy by changing the batch size with constant learning rate analysis.

In the study of the proposed model's performance when changing the batch sizes, while keeping the learning rate constant at 0.001, results indicate how batch size impacts the average accuracy of the model. When using the largest batch size of 2800, the model achieves the highest average accuracy of 99.11%. Reducing the batch size to 2000 results in a slight decrease in average accuracy to 98.96%. With a batch size of 1000, the model maintains

Learning rate	Average accuracy	Batch size
0.001	99.11	2800
0.005	98.84	2800
0.100	98.89	2800
0.200	98.91	2800
0.010	97.26	2800

Table 6. Analysis of proposed model on modifying the learning rate.

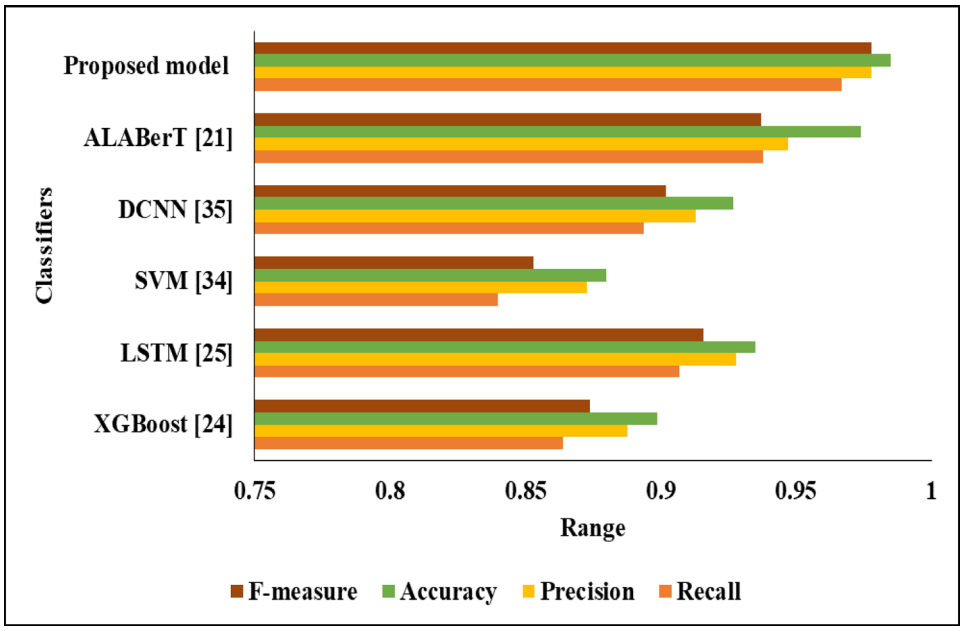


Fig. 7. Analysis of proposed model in first dataset.

a high average accuracy of 99.00%. Further reducing the batch size to 500 yields an average accuracy of 98.95%, while the smallest batch size of 100 results in the lowest average accuracy of 98.93%. These results suggest that larger batch sizes tend to provide better average accuracy, indicating improved performance. However, the decrease in accuracy with smaller batch sizes is relatively minor, suggesting that the model maintains robust performance across different batch sizes.

Validation analysis of proposed model by changing the learning rate

Table 6 explains the presentation of the projected model in terms of accuracy by changing the learning rate with constant batch size analysis.

In analyzing the performance of the proposed model with varying learning rates, the average accuracy was evaluated at a constant batch size of 2800. The model was trained using learning rates ranging from 0.001 to 0.200. Results indicate that a learning rate of 0.001 achieved the highest average accuracy of 99.11%. Increasing the learning rate to 0.005, 0.100, and 0.200 resulted in slightly lower average accuracies of 98.84%, 98.89%, and 98.91%, respectively. Conversely, reducing the learning rate to 0.010 significantly decreased the average accuracy to 97.26%. This demonstrates the critical influence of learning rate selection on model performance, highlighting the importance of optimizing this parameter for effective training and validation.

Comparative analysis of proposed model with existing techniques on considered dataset

The performance of the proposed model is compared with existing techniques such as XGBoost[24], LSTM[25], SVM[34], DCNN[35], and ALABerT[21] using various metrics. However, since the existing models use different datasets for sarcasm detection, this research implements the basic model and tests it on our two selected datasets. The average results are presented in Figs. 7 and 8.

In the experimental investigation, the performance of various techniques was evaluated using several metrics. The XGBoost[24] technique achieved a recall of 0.864, precision of 0.888, accuracy of 0.899, and an F1-score of 0.874. The LSTM[25] technique obtained a recall of 0.907, precision of 0.928, accuracy of 0.935, and an F1-score of 0.916. The SVM[34] technique resulted in a recall of 0.840, precision of 0.873, accuracy of 0.880, and an F1-score of 0.853. The DCNN[35] technique achieved a recall of 0.894, precision of 0.913, accuracy of 0.927, and an F1-score of 0.902. The ALABerT[21] technique showed a recall of 0.938, precision of 0.947, accuracy of 0.974, and an F1-score

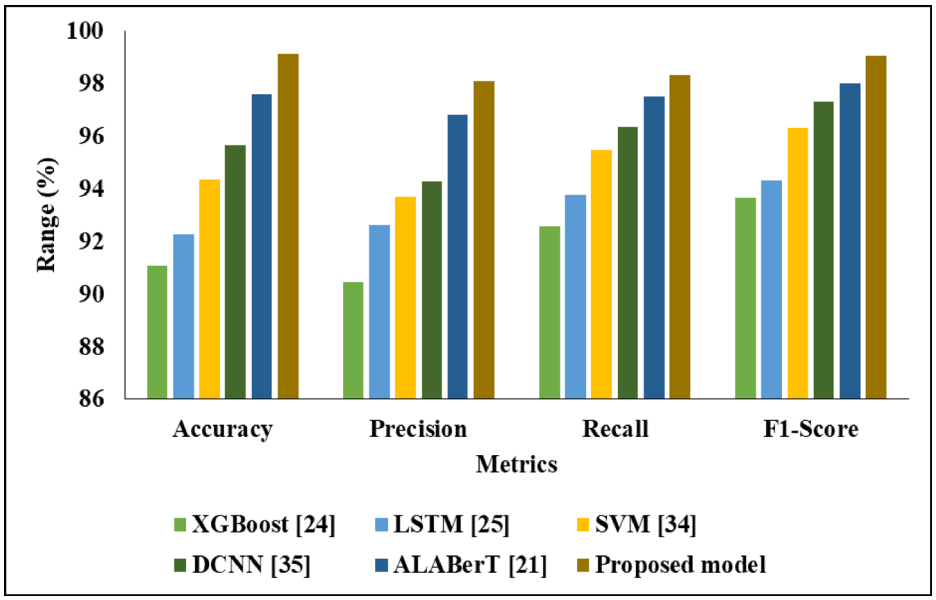


Fig. 8. Analysis of proposed model in second dataset.

Configuration	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Full Model (CNN + NGD + BERT + AM-BLSTM-GRU + ESRD)	99.13	98.09	98.33	99.06
Without NGD	97.85	96.10	95.50	96.80
Without ESRD Optimization	97.64	96.30	95.70	96.40
Without Attention Mechanism	96.90	95.80	94.90	95.30
Only BERT + CNN + BiGRU	96.40	94.70	94.10	94.40

Table 7. Ablation study of the proposed Model.

of 0.937. The proposed technique achieved the highest results with a recall of 0.967, precision of 0.978, accuracy of 0.985, and an F1-score of 0.978.

In the experimental analysis, the performance of various techniques was evaluated using several metrics. The XGBoost[24] technique achieved an accuracy of 91.08, precision of 90.46, recall of 92.60, and an F1-score of 93.66. The LSTM[25] technique showed an accuracy of 92.26, precision of 92.64, recall of 93.76, and an F1-score of 94.33. The SVM[34] technique achieved an accuracy of 94.37, precision of 93.69, recall of 95.48, and an F1-score of 96.33. The DCNN[35] technique resulted in an accuracy of 94.27, precision of 96.37, and recall of 97.33. The ALABerT[21] technique demonstrated an accuracy of 97.58, precision of 96.82, recall of 97.52, and an F1-score of 98.01. The proposed model achieved the highest performance with an accuracy of 99.13, precision of 98.09, recall of 98.33, and an F1-score of 99.06.

Ablation study

To assess the contribution of different components of the proposed model, ablation experiments were conducted by removing one component at a time and the results are presented in Table 7.

Removing NGD or ESRD optimization led to noticeable drops in accuracy and F1-score, highlighting their importance. The attention mechanism significantly improves recall and precision, proving its critical role in capturing key contextual information.

Generalization study

To evaluate the generalization performance, the model was tested on the Self-Annotated Reddit Sarcasm Corpus (SARC) without retraining. Table 8 provides Generalization Performance on Reddit Sarcasm Corpus.

The model achieved a strong F1-score of 95.30% on a completely unseen dataset, demonstrating its robustness and adaptability across platforms beyond Twitter and News headlines.

Discussion

BiGRU models offer bidirectional processing of sequences, enhancing context comprehension, but their bidirectionality results in increased computational complexity, leading to longer training times and higher resource demands. LSTM models effectively capture long-term dependencies but are vulnerable to gradient

Model	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
Proposed Model	96.70	95.10	95.40	95.30

Table 8. Generalization performance on reddit sarcasm Corpus.

issues in deep networks, while BiLSTM models, though robust for long sequences, risk overfitting in smaller datasets. GRU models, designed for simplicity, may struggle with complex sequential relationships due to their reduced gating mechanisms.

To mitigate these challenges, the proposed model integrates attention mechanisms, allowing for more precise feature selection and improving interpretability. Dropout regularization minimizes overfitting, and adaptive learning rates ensure stable training by dynamically adjusting learning rates, enhancing both performance and efficiency. While BiGRU models increase training time, their use is justified by the improvements in contextual understanding and classification accuracy, with adaptive learning and attention mechanisms optimizing the model's computational efficiency.

Managing noise from NGD semantic vectors is critical, as these vectors bring valuable external knowledge but can also introduce extraneous information, disrupting the original context of the input data. To address this, the model carefully balances the integration of NGD vectors by prioritizing relevant semantic features and filtering out less meaningful content. This refined approach preserves the integrity of the input, ensuring that the NGD information enhances rather than detracts from sentiment classification.

In conclusion, our model successfully balances computational complexity and noise management, leading to superior accuracy, precision, recall, and F1-score across various datasets. The combination of BiGRU, attention mechanisms, and NGD vector management advances the capabilities of sarcasm detection and sentiment analysis in challenging social media contexts.

Analysis of common misclassifications and edge cases

Despite the strong performance of our sarcasm detection model, certain challenges lead to misclassifications, particularly in subtle sarcasm, cultural variations, and context-dependent expressions. Statements lacking explicit cues, such as *"Oh great, another Monday"*, are often misclassified due to their ambiguous nature. Cultural nuances, where sarcasm varies by region (e.g., *"Brilliant idea, mate"* in British English), also pose difficulties. Additionally, context-dependent sarcasm, such as *"Love when my internet crashes"*, requires prior knowledge, making detection harder. Emojis and hashtags sometimes mislead the model, as sarcastic intent (e.g., "So happy…🙄") may be misinterpreted. Addressing these edge cases through sentiment intensity analysis, conversational context modeling, and multimodal integration can further enhance sarcasm detection accuracy.

Bias analysis and mitigation strategies

Our sarcasm detection model addresses key biases, including class imbalance, cultural bias, and platform-specific overrepresentation. To mitigate class imbalance, we applied SMOTE and optimized feature selection using ESRD. Cultural bias was reduced by incorporating multilingual datasets like iSarcasm. To counter platform bias, we validated the model on Twitter and Reddit sarcasm datasets for generalizability. Finally, contextual embeddings (NGD + BERT) were fine-tuned to prevent implicit bias in word associations. These strategies enhance fairness, ensuring a more balanced and adaptable sarcasm detection framework.

limitations

While the proposed model demonstrates superior performance in sarcasm detection, it also introduces several challenges, particularly in terms of computational complexity and model interpretability.

- Computational complexity: The use of BiGRU and attention mechanisms significantly improves the model's ability to capture both forward and backward dependencies in sequential data, which is crucial for sarcasm detection. However, these components come with increased computational costs, particularly in terms of memory usage and training time. BiGRU models, due to their bidirectional nature, are computationally expensive, and the integration of attention mechanisms further adds to the processing load. The ESRD optimizer, while effective in fine-tuning the model, also contributes to the increased training complexity. To address this, we employ adaptive learning rates and dropout regularization to stabilize training and reduce resource consumption, but the model remains relatively resource-intensive when applied to large datasets.
- Potential solutions: Future work could explore the use of more efficient architectures such as lightweight transformer models or pruning techniques that reduce the model's size and computational load without significantly compromising performance. Additionally, the use of distributed computing environments or GPU acceleration could alleviate the heavy resource demands during training.
- Model interpretability: Deep learning models, particularly those incorporating attention mechanisms and multiple layers like AM-BLSTM-GRU, often function as "black boxes," making it difficult to interpret how specific decisions are made. While attention mechanisms enhance the interpretability by focusing on key features, the complexity of combining multiple layers and optimizers can obscure the rationale behind certain predictions.
- Potential solutions: To improve interpretability, we plan to implement explainability techniques such as SHAP (SHapley Additive exPlanations) or LIME (Local Interpretable Model-agnostic Explanations), which provide insights into the model's decision-making process. These methods could help visualize the impact of different features on the model's predictions, offering more transparency in how the model detects sarcasm.

In conclusion, while the proposed model offers significant advantages in sarcasm detection, the trade-offs in computational complexity and interpretability need to be addressed in future iterations to make the model more practical and explainable for real-world applications.

Conclusion

This study demonstrates the significant impact of online reviews and comments on product sales and purchasing decisions. By analyzing product reviews through a deep learning-based sentiment analysis approach, we can enhance recommendation systems and improve the overall quality of insights for new users. The proposed model integrates a Convolutional Neural Network (CNN) with the Normalized Google Distance (NGD) for effective feature extraction, resulting in an advanced and practical deep learning model. Key components of the proposed approach include a sentiment analysis framework using an AM-BLSTM-GRU model and a comprehensive feature set.

Our method leverages an attention-based Bi-LSTM GRU to identify sarcasm in Twitter data, capitalizing on the vast amounts of available information. This combination enhances the model's durability and robustness, with the ESRD algorithm optimizing fine-tuning. Comparative analyses with existing LSTM methods show that our approach significantly improves accuracy by 5–10%, while maintaining or enhancing recall, precision, and F1-score. These results underscore the effectiveness of our approach in outperforming traditional benchmark representations.

Future work

Future work can focus on further refining the proposed model to enhance decision-making capabilities. Potential avenues for improvement include exploring additional feature extraction techniques and incorporating more diverse datasets to strengthen the model's adaptability and generalizability. Additionally, investigating real-time analysis and sentiment prediction could enhance the model's practical applications. Finally, integrating user feedback and adaptive learning mechanisms could provide a more personalized and dynamic sentiment analysis framework, further enhancing the utility and accuracy of the model in real-world scenarios.

Data availability

The datasets used and/or analyzed during the current study available from the corresponding author on reasonable request.

Received: 23 November 2024; Accepted: 19 June 2025
Published online: 02 July 2025

References

1. Goel, P., Jain, R., Nayyar, A., Singhal, S. & Srivastava, M. Sarcasm detection using deep learning and ensemble learning. *Multimedia Tools Appl.* **81** (30), 43229–43252 (2022).
2. Vinoth, D. & Prabhavathy, P. An intelligent machine learning-based sarcasm detection and classification model on social networks. *J. Supercomputing*. **78** (8), 10575–10594 (2022).
3. Krishnan, N., Rethnaraj, J. & Saravanan, M. Sentiment topic sarcasm mixture model to distinguish sarcasm prevalent topics based on the sentiment bearing words in the tweets. *J. Amb Intell. Hum. Comput.* **12**, 6801–6810 (2021).
4. Bharti, S. K. et al. Multimodal sarcasm detection: a deep learning approach. *Wirel. Commun. Mob. Comput.* **2022(1)**, 1653696 (2022).
5. Tan, Y. Y., Chow, C. O., Kanesan, J., Chuah, J. H. & Lim, Y. Sentiment analysis and sarcasm detection using deep multi-task learning. *Wireless Pers. Commun.* **129** (3), 2213–2237 (2023).
6. Saleem, H., Naeem, A., Abid, K. & Aslam, N. Sarcasm detection on twitter using deep handcrafted features. *J. Comput. Biomedical Inf.* **4** (02), 117–127 (2023).
7. Goel, P., Jain, R., Nayyar, A. & Singhal, S. And Srivastava M sarcasm detection using deep learning and ensemble learning. *Multimed Tools App*. **81**, 43229–43252 (2022).
8. Nayak, D. K. & Bolla, B. K. Efficient deep learning methods for sarcasm detection of news headlines. In: *Machine Learning and Autonomous Systems: Proceedings of ICMLAS 2021* (371–382). (Springer Nature Singapore, 2022).
9. Kumar, A. & Garg, G. Empirical study of shallow and deep learning models for sarcasm detection using context in benchmark datasets. *J. Ambient Intell. Humaniz. Comput.* **14** (5), 5327–5342 (2023).
10. Govindan, V. & Balakrishnan, V. A machine learning approach in analysing the effect of hyperboles using negative sentiment tweets for sarcasm detection. *J. King Saud University-Computer Inform. Sci.* **34** (8), 5110–5120 (2022).
11. Kumar, A., Sangwan, S. R., Singh, A. K. & Wadhwa, G. Hybrid deep learning model for sarcasm detection in Indian Indigenous Language using word-emoji embeddings. *ACM Trans. Asian Low-Resource Lang. Inform. Process.* **22** (5), 1–20 (2023).
12. Kamal, A. & Abulaish, M. Cat-bigru: Convolution and attention with bi-directional gated recurrent unit for self-deprecating sarcasm detection. *Cogn. Comput.* **14** (1), 91–109 (2022).
13. Godara, J., Aron, R. & Shabaz, M. Sentiment analysis and sarcasm detection from social network to train health-care professionals. *World J. Eng.* **19** (1), 124–133 (2022).
14. Misra, R. & Arora, P. Sarcasm detection using news headlines dataset. *AI Open.* **4**, 13–18 (2023).
15. Helal, N. A. et al. A contextual-based approach for sarcasm detection. *Sci. Rep.* **14**, 15415. https://doi.org/10.1038/s41598-024-65217-8 (2024).
16. Bhardwaj, S. & Prusty, M. R. BERT pre-processed deep learning model for sarcasm detection. *Natl. Acad. Sci. Lett.* **45** (2), 203–208 (2022).
17. Sharma, D. K., Singh, B., Agarwal, S., Kim, H. & Sharma, R. Sarcasm detection over social media platforms using hybrid auto-encoder-based model. *Electronics* **11** (18), 2844 (2022).
18. Vinoth, D. & Prabhavathy, P. Automated sarcasm detection and classification using hyperparameter tuned deep learning model for social networks. *Expert Syst.*, **39**(10), e13107. (2022).
19. Kumar, R. P., Mohan, G. B., Kakarla, Y., Jayaprakash, S. L., Sindhu, K. G., Chaitanya,T. V. S. S., … Krishna, N. H. (2023, July). Sarcasm detection in Telugu and Tamil:an exploration of machine learning and deep neural networks. In: *2023 14th International Conference on Computing Communication and Networking Technologies (ICCCNT)* (1–7).IEEE.

20. Sait, A. R. W. & Ishak, M. K. Deep learning with natural Language processing enabled sentimental analysis on sarcasm classification. *Comput. Syst. Sci. Eng.* **44** (3), 2553–2567 (2023).
21. Sukhavasi, V. & Dondeti, V. Effective automated transformer model-based sarcasm detection using multilingual data. *Multimedia Tools Appl.* **83** (16), 47531–47562 (2024).
22. Liu, H. et al. Sarcasm driven by sentiment: A sentiment-aware hierarchical fusion network for multimodal sarcasm detection. *Inform. Fusion*. **108**, 102353 (2024).
23. Prashanth, K. V. T., K., N. & Ramakrishnudu, T. *Sarcasm-based twee-tevel Stress Detection* e13534 (Expert Systems, 2024).
24. Ladoja, K. T. & Afape, R. T. Sarcasm detection in pidgin tweets using machine learning techniques. *Asian J. Res. Comput. Sci.* **17** (5), 212–221 (2024).
25. Gedela, R. T., Baruah, U. & Soni, B. Deep contextualised text representation and learning for sarcasm detection. *Arab. J. Sci. Eng.* **49** (3), 3719–3734 (2024).
26. Lora, S. K. et al. Ben-Sarc: A self-annotated corpus for sarcasm detection from Bengali social media comments and its baseline evaluation. *Nat. Lang. Process.*, 1–26 .
27. Aleryani, G. H., Deabes, W., Albishre, K. & Abdel-Hakim, A. E. Impact of Emoji exclusion on the performance of Arabic sarcasm detection models. *ArXiv Preprint arXiv* 240502195. (2024).
28. Liu, H., Yang, B. & Yu, Z. A multi-view interactive approach for multimodal sarcasm detection in social internet of things with knowledge enhancement. *Appl. Sci.* **14** (5), 2146 (2024).
29. Bousmaha, K. Z., Hamadouche, K., Djouabi, H. & Hadrich-Belguith, L. *Automatic Algerian Sarcasm Detection from Texts and Images* (ACM Transactions on Asian and Low-Resource Language Information Processing, 2024).
30. Galal, M. A., Yousef, A. H., Zayed, H. H. & Medhat, W. Arabic sarcasm detection: an enhanced fine-tuned Language model approach. *Ain Shams Eng. J.* **15** (6), 102736 (2024).
31. Shiwakoti, S. et al. Analyzing the dynamics of climate change discourse on twitter: A new annotated corpus and multi-aspect classification. In: *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)* (984–994) (2024).
32. Meng, J., Zhu, Y., Sun, S. & Zhao, D. Sarcasm detection based on BERT and attention mechanism. *Multimedia Tools Appl.* **83** (10), 29159–29178 (2024).
33. Ahire, L. K., Babar, S. D. & Mahalle, P. N. Mathematical analysis of different learning approaches on user behavior and contextual evaluation for sarcasm prediction.
34. Rajani, B., Saxena, S. & Kumar, B. S. Original research article detection of sarcasm in tweets using hybrid machine learning method. *J. Auton. Intell.*, **7**(4). (2024).
35. Sahu, G. A. & Hudnurkar, M. Metaheuristic-assisted deep ensemble technique for identifying sarcasm from social media data. *Int. J. Wireless Mobile Comput.* **26** (1), 25–38 (2024).
36. A, C. K. Twitter and reddit sentimental analysis dataset. Kaggle. Retrieved October 7, 2022, from (2019). https://www.kaggle.com/datasets/cosmos98/twitter-and-reddit-sentimental-analysis-dataset
37. Misra, R. News headlines dataset for sarcasm detection. Kaggle. Retrieved October 7, 2022, from (2019). https://www.kaggle.com/datasets/rmisra/news-headlines-dataset-for-sarcasm-detection
38. Muaad, A. Y., Jayappa Davanagere, H., Benifa, J. B., Alabrah, A., Naji Saif, M. A.,Pushpa, D., … Alfakih, T. M. (2022). Artificial Intelligence-Based Approach for Misogyny and Sarcasm Detection from Arabic Texts. *Comput. Intell. Neurosci.*,2022(1), 7937667.
39. Almuzaini, H. A. & Azmi, A. M. Impact of stemming and word embedding on deep learning-based Arabic text categorization. *IEEE Access.* **8**, 127913–127928. https://doi.org/10.1109/ACCESS.2020.3009217 (2020).
40. Devlin, J., Chang, M. W., Lee, K. & Toutanova, K. BERT: pretraining of deep bidirectional transformers for language understanding, *ArXiv* 181004805 Cs, (2019). http://arxiv.org/abs/1810.04805
41. Schick, T. & Schutze, H. Rare words: a major problem for contextualized embeddings and how to fix it by attentive mimicking. *Proc. AAAI Conf. Artif. Intell.* **34**, 8766–8774. https://doi.org/10.1609/aaai.v34i05.6403 (2020).
42. Nawaz, A., Asghar, S. & Naqvi, S. H. A. A segregational approach for determining aspect sentiments in social media analysis. *J. Supercomput.* **75**, 2584–2602 (2019).
43. Liu, W., Wang, Q., Zhu, Y. & Chen, H. GRU: optimization of NPI performance. *J. Supercomput.* **76** (5), 3542–3554 (2020).

Author contributions
All authors contributed equally.

Declarations

Competing interests
The authors declare no competing interests.

Ethics approval
The submitted work is original and has not been published elsewhere in any form or language.

Additional information
Correspondence and requests for materials should be addressed to R.V.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025